

Эпоха агентного программирования

v1.0 @ 02.2026

Written by: Andrei Leman & Claude Opus 4.6

Как автономные системы искусственного интеллекта переписывают правила разработки программного обеспечения — и что это значит для миллионов людей, зарабатывающих на жизнь написанием кода

Курсор мигает в терминале. Разработчик набирает одно предложение: «Добавь аутентификацию в API с использованием JWT-токенов, напиши тесты и обнови документацию». Затем откидывается на спинку кресла и наблюдает. В течение следующих четырёх минут ИИ-агент читает тридцать семь файлов, составляет план, пишет одиннадцать новых файлов, вносит изменения в шесть существующих, дважды запускает набор тестов — исправив упавшее утверждение со второй попытки, — коммитит изменения в Git и открывает pull request. Разработчик просматривает diff, одобряет его и переходит к следующей задаче.

Это не демонстрация. Это обычное утро вторника в феврале 2026 года.

I. Тихая революция

Между 2024 и 2026 годами в разработке программного обеспечения произошёл фундаментальный сдвиг, причём настолько постепенный, что многие практикующие специалисты заметили его лишь ретроспективно. Этим сдвигом стало не появление больших языковых моделей — они уже успели перевернуть индустрию благодаря инструментам автодополнения кода вроде GitHub Copilot, который к середине 2024 года генерировал примерно 40% нового кода в организациях, внедривших его. Настоящий сдвиг был тоньше и значительнее: ИИ-системы перестали быть инструментами, которыми пользуются разработчики, и начали становиться агентами, которыми разработчики руководят.

Это различие имеет огромное значение. Инструмент отвечает на единичный запрос — дополни эту строку, объясни эту функцию, предложи исправление этой ошибки. Агент преследует цель на протяжении множества шагов, принимая решения, восстанавливаясь после неудач и

координируя собственную работу. Разница аналогична пропасти между калькулятором и бухгалтером. Оба считают. Но только одному из них можно сказать «разберись с квартальными налогами» и предоставить самому выяснять детали.

К началу 2026 года только в экосистеме открытого исходного кода насчитывается не менее пятнадцати серьёзных ИИ-агентов для программирования, совокупно набравших более 500 000 звёзд на GitHub и ежедневно используемых сотнями тысяч разработчиков. Gemini CLI от Google лидирует с почти 96 000 звёзд. Zed, редактор на Rust со встроенной панелью агента, следует с 76 000. Cline, Codex, Claude Code и Aider группируются в диапазоне 40 000–58 000. За ними вторая волна — Goose, OpenCode, Warp, Continue, Crush — раздвигает границы возможностей терминальных агентов.

Это не исследовательские прототипы. Это продуктовые системы с моделями разрешений, сохранением сессий, изолированными средами выполнения и архитектурами плагинов. Вместе они представляют собой инфраструктуру новой парадигмы программирования.

II. Анатомия агента

Чтобы понять, почему этот момент отличается от предыдущих волн инструментов для разработчиков, полезно заглянуть внутрь механизма. Каждый крупный агент для программирования в 2026 году разделяет общий архитектурный скелет — паттерн, заимствованный из робототехники и исследований обучения с подкреплением, известный как цикл ReAct: Рассуждение, Действие, Наблюдение, Повторение.

Цикл работает следующим образом. Агент получает цель — скажем, «исправь падающий тест в `auth.test.js`». Он рассуждает о том, какая информация ему нужна, затем действует, вызывая инструмент: читает файл теста, изучает тестируемую функцию, ищет связанный код. Наблюдает результат, снова рассуждает и снова действует. Этот цикл продолжается — иногда три итерации, иногда тридцать — пока агент не определит, что цель достигнута или что он не может продолжить без участия человека.

Что делает этот обманчиво простой цикл мощным — это система инструментов под ним. Современные агенты несут наборы инструментов, которые показались бы абсурдными ещё два года назад. Crush, терминальный агент от Charmbracelet, поставляется с более чем двадцатью встроенными инструментами: чтение и запись файлов, выполнение команд оболочки с автоматическим переводом долгих процессов в фоновый режим, сопоставление шаблонов, поиск по содержимому, HTTP-запросы, веб-поиск через DuckDuckGo, кросс-репозиторийный поиск кода через Sourcegraph, диагностика через Language Server Protocol и выделенный подагент для веб-исследований, работающий с собственным изолированным набором инструментов. OpenCode, конкурент на Go, предлагает аналогичную широту возможностей плюс унифицированную систему патчей с обнаружением нечётких совпадений.

Codex от OpenAI сводит набор инструментов всего к двум базовым операциям — `apply_patch` и `local_shell_call` — но оборачивает их в самую изощрённую систему песочницы в экосистеме.

Инструменты — это руки агента. Языковая модель — его мозг. А цикл ReAct — нервная система, соединяющая их. Но настоящий инженерный вызов — то, что отделяет впечатляющую демонстрацию от надёжного повседневного инструмента — заключается во всём остальном: системах разрешений, управлении контекстом, восстановлении после ошибок и тонком искусстве понимания, когда следует остановиться.

Проблема контекста

Большие языковые модели работают в рамках фиксированного контекстного окна — объёма текста, который они могут рассматривать одновременно. Для большинства моделей в начале 2026 года он составляет от 120 000 до 200 000 токенов, что примерно эквивалентно роману в 400 страниц. Модели Gemini от Google расширяют этот предел до одного миллиона токенов — достаточно, чтобы удержать в памяти целую кодовую базу среднего размера.

Это звучит щедро, пока не задумаешься о том, что агент на самом деле делает во время сложной задачи. Каждый прочитанный файл, каждый обработанный результат поиска, каждый изученный вывод инструмента — всё это накапливается в контекстном окне. Одно чтение файла может потребить 2 000 токенов. Поиск, вернувший двадцать совпадений, обходится в 3 000. Полный цикл «рассуждение — действие — наблюдение» сжигает от 3 000 до 8 000 токенов. Агент, работающий над нетривиальной функциональностью, может исчерпать окно в 120 000 токенов за пятнадцать-двадцать циклов.

Решения этой проблемы обнажают философские различия между проектами. Gemini CLI обходит её грубой силой — окно в миллион токенов редко заполняется. Claude Code реализует изощрённую компактификацию: когда контекст достигает порогового значения, система резюмирует более ранние сегменты разговора и отбрасывает сырой текст, сохраняя смысл и высвобождая пространство. Aider, пионер на Python, избрал, пожалуй, самый элегантный подход: карты репозитория на основе tree-sitter. Вместо чтения целых файлов Aider использует разбор абстрактного синтаксического дерева для построения структурной карты кодовой базы — сигнатуры функций, иерархии классов, связи импортов — которая занимает лишь малую долю пространства, сохраняя при этом информацию, наиболее часто необходимую агенту.

OpenCode и Crush реализуют автоматическое резюмирование, сжимая историю сессии по мере её роста. Codex использует скользящее окно из трёх-пяти последних взаимодействий, позволяя более раннему контексту уходить. Каждый подход по-своему балансирует между полнотой и эффективностью, и ни один не является безусловно лучшим. Проблема контекста остаётся одной из самых сложных нерешённых задач в проектировании агентов.

Вопрос разрешений

Когда вы даёте агенту возможность выполнять команды оболочки, записывать файлы и отправлять HTTP-запросы, вы вручаете ему ключи от вашей среды разработки. Вопрос о том, сколько автономии предоставить — и как её ограничить — породил наибольшее расхождение в философии проектирования по всей экосистеме.

Codex от OpenAI занимает наиболее радикальную позицию: песочница на уровне операционной системы. На macOS он использует фреймворк Apple Seatbelt для создания изолированной среды с доступом только на чтение вокруг среды выполнения агента, с настраиваемыми исключениями для определённых директорий. На Linux применяется Landlock — модуль безопасности на уровне ядра. В режиме «полной автоматизации» по умолчанию сетевой доступ полностью заблокирован, а запись файлов ограничена директорией проекта. Агент физически не может выйти в интернет или затронуть файлы за пределами своей песочницы, независимо от того, что решит языковая модель.

Это принципиально иная модель безопасности по сравнению с тем, что предлагают все остальные агенты. Crush реализует многоуровневую систему разрешений — чёрные списки команд, белые списки безопасных операций, обязательное чтение перед редактированием, проверку времени модификации — но всё это носит рекомендательный характер. Они полагаются на то, что агент будет соблюдать правила. Песочница Codex обеспечивается ядром операционной системы. Агент не может нарушить её точно так же, как обычный пользовательский процесс не может записать данные в домашнюю директорию другого пользователя.

Claude Code избирает ещё один подход: систему хуков жизненного цикла с четырнадцатью различными событиями — от `SessionStart` до `PreToolUse` и `Stop` — которые позволяют внешним скриптам перехватывать, модифицировать или блокировать любое действие агента. Это наиболее гибкая система, позволяющая организациям реализовывать сколь угодно сложные политики, но она требует, чтобы кто-то эти политики написал.

Спектр простирается от Aider, у которого вообще нет системы разрешений (он автоматически коммитит изменения в Git, используя систему контроля версий как страховочную сеть), до ядерной изоляции Codex. Большинство инструментов располагаются где-то посередине, предлагая флаг `--yo1o` или его аналог для разработчиков, которые хотят максимальной скорости и готовы принять связанные с этим риски.

III. Кембрийский взрыв

Разнообразие подходов в начале 2026 года поразительно. Взять хотя бы выбор языков программирования. Два года назад большинство ИИ-инструментов писалось на Python или TypeScript — языках, наиболее близких к экосистеме машинного обучения. Сегодня наиболее технически амбициозные новые агенты написаны на Go и Rust.

OpenCode и Crush написаны на Go с использованием фреймворка Bubble Tea для терминальных интерфейсов — библиотеки, ставшей де-факто стандартом для богатых терминальных приложений в экосистеме Go. Текущая реализация Codex выполнена на Rust с использованием Ratatui для TUI. Сдвиг в сторону системных языков отражает зрелость приоритетов: время запуска, эффективность использования памяти и возможность реализации песочницы на уровне ОС имеют значение, когда инструмент запускается сотни раз в день.

Но настоящий кембрийский взрыв происходит в пространстве функциональности. Каждый проект занял свою отличительную территорию:

Aider стал пионером концепции карты репозитория — используя tree-sitter, генератор парсеров, изначально созданный для подсветки синтаксиса в редакторах, для построения абстрактного синтаксического дерева всей кодовой базы. Это даёт агенту структурное понимание кода без необходимости читать каждый файл. Aider также рассматривает Git как полноценную систему управления сессиями: каждое изменение, вносимое агентом, автоматически коммитится, создавая гранулярную историю отмен, по которой разработчики могут перемещаться стандартными командами Git. Это был первый инструмент, продемонстрировавший, что ИИ-агент может быть настоящим напарником по программированию, а не изощёренным движком автодополнения.

Goose, созданный компанией Block (ранее Square), принял радикальное архитектурное решение: расширения являются MCP-серверами. Вместо построения монолитной системы инструментов Goose рассматривает каждую возможность — файловые операции, веб-поиск, доступ к базам данных — как внешний сервис, взаимодействующий через Model Context Protocol. Это делает Goose, пожалуй, наиболее расширяемым агентом в экосистеме, ценой более сложной настройки и потенциальной задержки от межпроцессного взаимодействия.

Gemini CLI использовал инфраструктурное преимущество Google — контекстное окно в один миллион токенов и щедрый бесплатный тариф в шестьдесят запросов в минуту — чтобы привлечь крупнейшее сообщество в этом пространстве. Его подход к проблеме контекста по сути состоит в том, чтобы сделать её неактуальной: когда можно удерживать в памяти всю кодовую базу, карты репозитория и алгоритмы компактификации не нужны.

Cline, работающий как расширение VS Code, а не как CLI, раздвинул границы того, что означает «автономный» на практике. Он может выполнять многошаговые рабочие процессы,

автоматизировать взаимодействие с браузером через Playwright и даже использовать компьютерное зрение для работы с графическими интерфейсами. Его экосистема форков — Roo Code, Kilo Code — свидетельствует о том, что модель расширений VS Code может быть столь же плодородной почвой для разработки агентов, как и терминал.

Zed, редактор на Rust, представляет совершенно иной тезис: агент должен быть встроен в редактор, а не работать рядом с ним. Его панель агента предоставляет разговорный интерфейс внутри среды редактирования, а его Agent Client Protocol (ACP) позволяет сторонним агентам интегрироваться напрямую. С 76 000 звёзд и системой совместного редактирования Zed предполагает, что будущее агентов для программирования может быть вовсе не в терминале.

Стандарт MCP

Если есть одно техническое достижение, определяющее ландшафт агентов 2026 года, то это Model Context Protocol. MCP, первоначально предложенный Anthropic в конце 2024 года, стал универсальным стандартом расширяемости для ИИ-агентов. Каждый крупный инструмент в экосистеме либо поддерживает его, либо объявил о планах сделать это.

MCP — это, по сути, стандартизированный способ для ИИ-агента обнаруживать и использовать внешние инструменты. MCP-сервер предоставляет набор возможностей — инструменты (функции, которые агент может вызывать), ресурсы (данные, которые агент может читать) и промпты (шаблоны, которые агент может использовать) — через транспортный уровень, который может быть как простым, как стандартные каналы ввода/вывода, так и сложным, как HTTP с server-sent events.

Значимость MCP заключается не в его технической изощрённости — это относительно простой протокол JSON-RPC — а в его универсальности. До MCP каждый агент имел собственный формат плагинов, собственный механизм регистрации инструментов, собственный способ расширения возможностей. Инструмент, написанный для Aider, нельзя было использовать в Claude Code. Расширение Goose было бесполезно в OpenCode. MCP изменил это. Единый MCP-сервер — скажем, предоставляющий доступ к трекеру задач Jira или кластеру Kubernetes — теперь может использоваться любым MCP-совместимым агентом без модификации.

Crush реализует полную спецификацию MCP по трём транспортным уровням (stdio, HTTP и server-sent events) с конечным автоматом, управляющим жизненным циклом соединения, и проверкой разрешений для каждого инструмента. Codex идёт дальше: он может не только подключаться к MCP-серверам как клиент, но и предоставлять себя в качестве MCP-сервера, что открывает возможность для так называемых мета-агентных архитектур — систем, в которых один ИИ-агент использует другого ИИ-агента как инструмент. Насколько можно судить по открытым источникам, это первая реализация рекурсивной композиции агентов в продуктивном инструменте для программирования.

Последствия значительны. Если агенты могут использовать других агентов как инструменты, потолок того, что может быть достигнуто одной командой, резко возрастает. Разработчик мог бы, в принципе, поручить одному агенту координировать команду специализированных подагентов — одного для фронтенда, одного для бэкенда, одного для тестирования, одного для документации — каждый из которых работает в собственной изолированной среде со своим контекстным окном. Это пока не стало распространённой практикой, но инфраструктура для этого существует уже сегодня.

Преимущество LSP

Один из наиболее недооценённых дифференцирующих факторов в экосистеме агентов — интеграция с Language Server Protocol. LSP, изначально разработанный Microsoft для Visual Studio Code, предоставляет стандартизированный интерфейс для языково-специфичного интеллекта: диагностика (обнаружение ошибок и предупреждений), навигация по коду (переход к определению, поиск ссылок), автодополнение и рефакторинг.

Только два из пятнадцати крупных агентов — Crush и OpenCode — имеют глубокую интеграцию с LSP, и разница, которую она даёт, существенна. Когда Crush записывает или модифицирует файл, он автоматически запрашивает у соответствующего языкового сервера диагностику. Если языковой сервер сообщает об ошибке типа или синтаксической проблеме, агент видит это немедленно — не после запуска компилятора, не после того как разработчик заметит, а как часть самой операции записи. Инструмент `references` в Crush может найти каждое использование символа по всей кодовой базе, обращаясь к языковому серверу, который понимает код семантически, а не полагается на текстовое сопоставление.

В этом разница между агентом, который обращается с кодом как с текстом, и агентом, который обращается с кодом как с кодом. Агент с интеграцией LSP знает, что переименование функции требует обновления каждого места вызова, что несоответствие типов в одном файле вызовет ошибку компиляции в другом, что неиспользуемый импорт следует удалить. Агент без неё — это, по сути, очень изощённый текстовый редактор, который случайно понимает концепции программирования.

Редкость интеграции с LSP — только два из пятнадцати инструментов — вероятно, отражает инженерную сложность задачи. Поддержка LSP означает управление множеством процессов языковых серверов, обработку асинхронной диагностики, работу с падениями и перезапусками серверов и отображение между файловыми операциями агента и документной моделью языкового сервера. На языке программной инженерии это «сложная задача». Но это также, пожалуй, единственная функция, которая с наибольшей вероятностью отделит надёжных агентов от ненадёжных по мере развития технологии.

IV. Человек в контуре

Самый интересный вопрос об ИИ-агентах для программирования — не технический. Он социологический: как меняются отношения между разработчиком и его кодом, когда автономная система может писать, тестировать и развёртывать программное обеспечение от его имени?

Ранние данные свидетельствуют о сдвиге от написания к руководству. Разработчики, использующие агентов, сообщают, что тратят меньше времени на набор кода и больше — на его рецензирование: чтение diff-ов, оценку архитектурных решений, анализ покрытия тестами. Набор навыков мигрирует от реализации к спецификации: способность чётко описать, что вы хотите, декомпозировать сложную цель на достижимые шаги, распознать, когда результат работы агента содержит неочевидную ошибку.

Это не совсем ново. Старшие инженеры всегда тратили больше времени на проектирование и рецензирование, чем на набор текста. Новое в том, что этот сдвиг происходит раньше в карьере и более полно. Младший разработчик с хорошо настроенным агентом может производить работающий код с такой скоростью, которая была бы невозможна два года назад. Узкое место смещается от «можешь ли ты это написать?» к «можешь ли ты оценить, правильно ли это?»

Проблема спецификации

В эпохе агентов есть глубокая ирония: чем лучше агенты пишут код, тем важнее становится для людей точно формулировать, чего они хотят. Расплывчатая инструкция коллеге-человеку может породить разумную интерпретацию, основанную на общем контексте, нормах команды и профессиональном суждении. Та же расплывчатая инструкция агенту порождает... нечто. Оно будет синтаксически корректным, вероятно, пройдёт тесты, которые агент для него напишет, и может соответствовать или не соответствовать тому, что вам на самом деле было нужно.

Это привело к появлению того, что можно назвать «пром프트-инженерией для кода» — хотя практикующие специалисты склонны избегать этого термина, предпочитая «спецификацию» или просто «чёткое описание того, что вы хотите». Наиболее эффективные пользователи агентов выработали привычки, удивительно похожие на практики формальной спецификации, которые информатика пропагандировала десятилетиями: формулирование предусловий и постусловий, явное определение граничных случаев, описание не только того, что код должен делать, но и почему.

Файлы инструкций проекта, которые поддерживает каждый крупный агент — `CLAUDE.md` для Claude Code, `AGENTS.md` для Codex и Crush, `GEMINI.md` для Gemini CLI, файлы `.aider*` для Aider — являются, по сути, облегчёнными документами спецификации. Они описывают соглашения по написанию кода, архитектурные решения, требования к тестированию и

предметно-специфичные ограничения. Хорошо поддерживаемый файл инструкций проекта может кардинально улучшить качество результатов агента, поскольку предоставляет контекст, который коллега-человек усвоил бы за месяцы работы над проектом.

Доверие и верификация

Системы разрешений, описанные ранее — песочницы, чёрные списки, системы хуков, запросы на подтверждение — являются техническими решениями фундаментально человеческой проблемы: насколько вы доверяете агенту?

Ответ на практике варьируется чрезвычайно. Некоторые разработчики запускают Crush в режиме `--yo!o`, отключая все проверки разрешений, потому что доверяют своей истории Git как страховочной сети. Другие настраивают систему хуков Claude Code так, чтобы требовать явного одобрения каждой записи файла, обращаясь с агентом как с ненадёжным подрядчиком, чью работу необходимо проверить, прежде чем она затронет кодовую базу. Модель песочницы Codex предлагает средний путь: агент может делать что угодно в пределах своей изолированной среды, но физически не может повлиять ни на что за её пределами.

Вопрос доверия обостряется по мере роста возможностей агентов. Агент, который может только редактировать файлы, относительно безопасен — худшее, что он может сделать, это написать плохой код, который выявит рецензирование. Агент, который может выполнять команды оболочки, отправлять HTTP-запросы и взаимодействовать с внешними сервисами, имеет значительно больший радиус поражения. Агент, который может порождать подагентов, каждый со своим набором инструментов, вносит комбинаторное расширение возможных действий, которое ни один человек не может полностью предвидеть.

Вот почему подход Codex к песочнице — обеспечиваемый ядром операционной системы, а не собственной дисциплиной агента — может оказаться наиболее важным архитектурным решением в экосистеме. Это единственный подход, который предоставляет гарантии безопасности, а не надежды на безопасность.

V. Экономика внимания

Существует ресурс более ограниченный, чем вычислительная мощность, более ценный, чем токены, и менее изученный, чем любой технический параметр в экосистеме агентов: внимание разработчика.

Каждое взаимодействие с агентом — каждый написанный промпт, каждый просмотренный diff, каждое одобренное разрешение — стоит внимания. Обещание агентов в том, что они снижают совокупные затраты внимания на разработку программного обеспечения, автономно выполняя

рутинную работу. Риск в том, что они лишь перераспределяют его: меньше времени на написание кода, больше — на надзор за агентом, рецензирование его результатов и исправление его ошибок.

Инструменты, получившие наибольшее распространение, — это те, которые наиболее эффективно управляют этой экономикой внимания. Стратегия автоматических коммитов Aider — образец управления вниманием: коммита каждое изменение в Git, он даёт разработчикам механизм отмены с нулевой стоимостью. Если агент выдал плохой результат, `git revert` быстрее, чем чтение diff-а. Это освобождает разработчика для подхода «попробуй и посмотри» вместо тщательной оценки каждого предложенного изменения перед его применением.

Система фоновых задач Crush решает другое узкое место внимания. Когда команда оболочки выполняется более шестидесяти секунд — набор тестов, процесс сборки, миграция базы данных — Crush автоматически переводит её в фоновый режим и продолжает работать над другими задачами. Разработчик может проверить фоновые задачи в любой момент, но не вынужден ждать. Эта, казалось бы, небольшая функция устраняет один из наиболее распространённых источников напрасно потраченного внимания в разработке с помощью агентов: созерцание терминала во время выполнения длительного процесса.

Система подагентов Claude Code (инструмент Task) решает проблему на более высоком уровне. Когда сложная задача требует исследования нескольких частей кодовой базы, основной агент может порождать специализированных подагентов — каждый со своим изолированным контекстным окном — для параллельного исследования. Результаты возвращаются основному агенту, который синтезирует их без необходимости для разработчика управлять процессом исследования. Внимание разработчика требуется только в точках принятия решений, а не во время сбора информации.

Это не просто удобные функции. Они воплощают фундаментальное понимание экономики сотрудничества человека и ИИ: ценность агента измеряется не тем, сколько кода он может написать, а тем, как мало внимания требуется для получения корректных результатов.

Парадокс автономии

В сердце эпохи агентов лежит парадокс. Разработчики хотят, чтобы агенты были более автономными — выполняли больше шагов без прерывания, принимали больше решений самостоятельно, требовали меньше надзора. Но они также хотят, чтобы агенты были более предсказуемыми — никогда не вносили неожиданных изменений, всегда объясняли свои рассуждения, останавливались и спрашивали при неопределённости.

Эти желания находятся в противоречии. Агент, который останавливается и спрашивает о каждом неоднозначном решении, безопасен, но медлителен. Агент, который действует решительно и делает наилучшее предположение, быстр, но иногда ошибается так, что

исправление обходится дорого. Оптимальный баланс зависит от задачи, разработчика, кодовой базы и ставок.

Экосистема пришла к приблизительному консенсусу: агенты должны быть автономны в хорошо понятных операциях (чтение файлов, запуск тестов, поиск по коду) и консультативны в последственных (удаление файлов, изменение конфигурации, отправка в удалённые репозитории). Но граница между «хорошо понятным» и «последственным» сама по себе является проектным решением, которое разные инструменты проводят по-разному.

Трёхрежимная система Codex — только чтение, запись в рабочую область и полный доступ с повышенным риском — делает это явным. В режиме только чтения агент может исследовать, но не модифицировать. В режиме записи в рабочую область он может изменять файлы в директории проекта. В режиме полного доступа все ограничения снимаются. Разработчик выбирает уровень автономии перед началом сессии, принимая осознанное решение о компромиссе между доверием и скоростью.

Эта модель градуированной автономии может оказаться наиболее важным UX-паттерном, возникшим в эпоху агентов. Она признаёт, что доверие не бинарно — оно контекстуально, зависит от задачи и зарабатывается со временем.

VI. Что дальше

Траектория развития ИИ-агентов для программирования указывает на несколько направлений, которые, вероятно, преобразят разработку программного обеспечения в ближайшие два-три года.

Мультиагентная оркестрация. Инфраструктура для координации агентов друг с другом — MCP как стандарт коммуникации, порождение подагентов, паттерны «агент как сервер» — уже существует. Следующий шаг — системы, в которых несколько специализированных агентов совместно работают над одной задачей, каждый привнося предметную экспертизу, которой лишён агент-универсал. Навык `babysit-pr` в Codex, который автономно отслеживает pull request-ы, классифицирует сбои CI и применяет исправления, — ранний пример того, как это выглядит на практике: агент, который действует не как помощник разработчика, а как независимый участник рабочего процесса разработки.

Более глубокое семантическое понимание. Разрыв между агентами с интеграцией LSP и без неё будет увеличиваться. По мере того как языковые модели совершенствуются в рассуждениях о структуре кода, агенты, способные предоставить им семантическую информацию — иерархии типов, графы вызовов, связи зависимостей — будут давать кардинально лучшие результаты, чем те, что ограничены пониманием на уровне текста. Карты

репозитория на основе tree-sitter, впервые предложенные Aider, были первым шагом. Полная интеграция с LSP, реализованная в Crush, — вторым. Третий шаг — пока не достигнутый ни одним инструментом — это глубокая интеграция с системами сборки, менеджерами пакетов и профилировщиками времени выполнения, дающая агентам не только структурное, но и поведенческое понимание модифицируемого кода.

Персистентная память и обучение. Нынешние агенты в значительной степени не сохраняют состояние между сессиями. Они могут хранить историю разговоров, но не учатся на опыте в каком-либо значимом смысле. Разработчик, исправивший ошибку агента в понедельник, скорее всего, увидит ту же ошибку во вторник. Файлы инструкций проекта (`CLAUDE.md` , `AGENTS.md`) — это ручной обходной путь: разработчик кодирует извлечённые уроки в виде явных инструкций. Но естественная эволюция ведёт к агентам, которые автоматически накапливают знания, специфичные для проекта: какие паттерны предпочитает разработчик, какие подходы уже терпели неудачу, какие части кодовой базы хрупки и требуют особой осторожности.

Интеграция формальной верификации. По мере того как агенты берут на себя всё больше ответственности за корректность кода, спрос на автоматизированную верификацию будет расти. Сегодняшние агенты полагаются преимущественно на наборы тестов для валидации своей работы — они пишут код, запускают тесты и итерируют, пока тесты не пройдут. Но наборы тестов — это неполные спецификации. Они проверяют конкретные случаи, а не общие свойства. Интеграция инструментов формальной верификации — тестирования на основе свойств, проверки моделей, статического анализа с математическими гарантиями — в рабочие процессы агентов обеспечила бы уровень уверенности, недостижимый одним лишь тестированием.

Конец файла как единицы работы. Нынешние агенты работают преимущественно на уровне файлов: они читают файлы, редактируют файлы, создают файлы. Но программное обеспечение организовано не по файлам — оно организовано по функциональностям, модулям и аспектам, пересекающим границы файлов. Агенты, способные рассуждать о сквозных аспектах — «добавь логирование в каждый эндпоинт API», «убедись, что все запросы к базе данных используют параметризованные выражения», «обнови каждый компонент, использующий старый API аутентификации» — откроют категорию задач, с которыми нынешние инструменты справляются плохо. Интеграция с Sourcegraph, присутствующая и в OpenCode, и в Crush, указывает в этом направлении: кросс-репозиторийный поиск кода — предпосылка для сквозных модификаций.

Вопрос о рабочей силе

Ни одно обсуждение ИИ-агентов для программирования не будет полным без обращения к вопросу, который тенью нависает над каждым разговором об этой технологии: что будет с программистами?

Исторические данные об автоматизации и занятости более нюансированы, чем склонны признавать и оптимисты, и пессимисты. Появление электронных таблиц не уничтожило бухгалтеров — оно уничтожило счетоводов и создало новую категорию финансовых аналитиков, способных делать то, что раньше было невозможно. Появление систем автоматизированного проектирования не уничтожило архитекторов — оно уничтожило чертёжников и позволило достичь архитектурной сложности, которую ручное черчение никогда не могло бы обеспечить.

Закономерность устойчива: автоматизация устраняет рутинные компоненты квалифицированного труда, усиливая при этом его творческие и оценочные компоненты. Эпоха агентов, по-видимому, следует этой закономерности. Задачи, с которыми агенты справляются лучше всего — реализация хорошо специфицированных функций, написание шаблонного кода, исправление простых ошибок, написание тестов для существующего кода — это именно те задачи, которые опытные разработчики находят наименее увлекательными. Задачи, с которыми агенты справляются плохо — понимание неоднозначных требований, принятие архитектурных решений, оценка компромиссов между конкурирующими подходами к проектированию, навигация в организационной политике — это задачи, определяющие работу старшего инженера.

Это не означает, что переход будет безболезненным. Спрос на чисто реализационную работу — перевод чёткой спецификации в работающий код — вероятно, снизится. Роли младших разработчиков, состоявшие преимущественно из реализации под пристальным руководством, будут эволюционировать. Новая точка входа в профессию может выглядеть не как «напиши эту функцию», а как «оцени, соответствует ли этот сгенерированный агентом код спецификации, и если нет, выясни почему».

Будет ли это чистым позитивом, зависит от факторов, которые трудно предсказать: насколько быстро образовательные учреждения адаптируют свои программы, насколько эффективно организации перестроят свои команды и создаст ли повышение производительности, обеспечиваемое агентами, достаточно нового спроса на программное обеспечение, чтобы компенсировать сокращение трудозатрат на единицу продукции. Оптимистичный сценарий — что агенты сделают разработку программного обеспечения доступной для значительно более широкого круга людей, создавая спрос, многократно превышающий нынешний уровень — правдоподобен. Как и пессимистичный — что агенты сконцентрируют прирост производительности среди меньшего числа высококвалифицированных разработчиков, выхолащивая середину профессии.

Что очевидно — это смещение значимых навыков. Способность писать синтаксически корректный код на конкретном языке становится менее ценной. Способность рассуждать о системах, точно специфицировать поведение, оценивать корректность, принимать архитектурные решения в условиях неопределённости — всё это становится более ценным.

Эпоха агентов не устраняет потребность в человеческом суждении. Она делает человеческое суждение узким местом.

VII. Новые отношения с кодом

Пожалуй, наиболее глубокое изменение эпохи агентов — не техническое и не экономическое, а психологическое. На протяжении пятидесяти лет программирование было актом непосредственного творения — разработчик думает, набирает текст и видит, как его мысли материализуются в работающее программное обеспечение. В этом процессе есть интимность, ощущение ремесла, которое многие разработчики описывают как главную притягательность своей профессии.

Агенты вводят прослойку между разработчиком и кодом. Разработчик по-прежнему думает, но больше не набирает — или, точнее, набирает инструкции, а не реализации. Код, который получается в результате, не вполне его, не вполне агента, а нечто, произведённое в сотрудничестве. Одни разработчики находят это освобождающим: избавленные от механики реализации, они могут сосредоточиться на задачах, которые их действительно волнуют. Другие находят это отчуждающим: код ощущается не столько как их творение, сколько как нечто, что они одобрили.

Это напряжение заметно в дизайне самих инструментов. Подход Aider — автоматический коммит каждого изменения с указанием разработчика в качестве автора — имплицитно утверждает, что сгенерированный агентом код является кодом разработчика, произведённым с помощью изощрённого инструмента, принципиально не отличающегося от функций рефакторинга IDE. Призрачные коммиты Codex — невидимые снимки, не появляющиеся в публичной истории Git — предполагают иную философию: работа агента носит предварительный характер, черновик, который становится реальностью лишь тогда, когда разработчик явно его принимает.

Истина, как обычно, где-то посередине. Сгенерированный агентом код не принадлежит полностью ни разработчику, ни машине. Это новая категория артефакта, произведённого в сотрудничестве, не имеющем точного исторического прецедента. Ближайшая аналогия — отношения между архитектором и инженером-конструктором: архитектор задаёт видение, инженер обеспечивает его устойчивость, а получившееся здание принадлежит обоим и никому.

VIII. Заключение: терминал как мастерская

В феврале 2026 года терминал — этот аскетичный прямоугольник моноширинного текста, бывший основным рабочим пространством программиста с 1970-х годов — переживает свою наиболее значительную трансформацию за десятилетия. Мигающий курсор, некогда ожидавший нажатий клавиш, теперь ожидает инструкций. Командная строка, некогда выполнявшая единичные операции, теперь оркестрирует многошаговые рабочие процессы. Оболочка, некогда соединявшая человека с операционной системой, теперь соединяет человека с интеллектом.

Полтора десятка инструментов, конкурирующих в этом пространстве — от массово популярного Gemini CLI от Google до тщательно выверенного Crush от Charmbracelet, от ориентированного на безопасность Codex от OpenAI до богатого хуками Claude Code от Anthropic — это не просто продукты. Это эксперименты в новом режиме взаимодействия человека и компьютера, каждый из которых проверяет свою гипотезу о том, как люди и ИИ-системы должны сотрудничать в сложной, творческой, изматывающей и глубоко вознаграждающей работе по созданию программного обеспечения.

Эпоха агентного программирования — не будущее состояние. Это настоящее, наступающее неравномерно, понимаемое неполно и стремительно эволюционирующее. Преуспеют в ней не те разработчики, которые пишут больше всего кода, а те, кто наиболее мудро им руководят — кто способен взглянуть на результат работы агента и увидеть не только то, что он сделал, но и то, что он должен был сделать иначе. В этом смысле эпоха агентов не умаляет ремесло программирования. Она обнажает то, чем это ремесло всегда было на самом деле: не набор текста, а мышление.

Автор признаёт использование ИИ-агентов для программирования при подготовке данной рукописи, что представляется вполне уместным.

Врезка 1: Цифры

Показатель	Значение (февраль 2026)
Крупных ИИ-агентов для программирования с открытым исходным кодом	~15
Совокупное количество звёзд на GitHub	>500 000

Показатель	Значение (февраль 2026)
Наиболее популярный инструмент	Gemini CLI (~96 тыс.)
Языки реализации агентов	Go, Rust, TypeScript, Python
Поддерживаемых провайдеров LLM (лучший инструмент)	10+ (Codex)
Наибольшее контекстное окно	1 млн токенов (Gemini)
Наибольшее количество встроенных инструментов	20+ (Crush)
Событий хуков жизненного цикла (максимум)	14 (Claude Code)
Реализаций песочницы на уровне ОС	1 (Codex: Seatbelt + Landlock)
Инструментов с глубокой интеграцией LSP	2 (Crush, OpenCode)
Универсальный стандарт расширяемости	MCP (Model Context Protocol)

Врезка 2: Глоссарий

- **Цикл ReAct** — цикл «Рассуждение — Действие — Наблюдение»; стандартный архитектурный паттерн агента
- **MCP** — Model Context Protocol; стандартизированный интерфейс для коммуникации между агентом и инструментами
- **LSP** — Language Server Protocol; стандартизированный интерфейс для языково-специфичного интеллекта в работе с кодом
- **Контекстное окно** — максимальный объём текста, который языковая модель может обработать одновременно
- **Компактификация** — резюмирование более ранних частей разговора для высвобождения пространства контекстного окна
- **Карта репозитория** — представление кодовой базы в виде абстрактного синтаксического дерева для эффективного использования контекста
- **Песочница** — изоляция на уровне ОС, ограничивающая доступ агента к системным ресурсам
- **Подагент** — вторичный ИИ-агент, порождаемый основным агентом для выполнения подзадачи

- **Призрачный коммит** — невидимый снимок Git, используемый для отката без засорения истории
- **TUI** — Terminal User Interface; графический интерфейс, отрисовываемый в текстовом терминале